

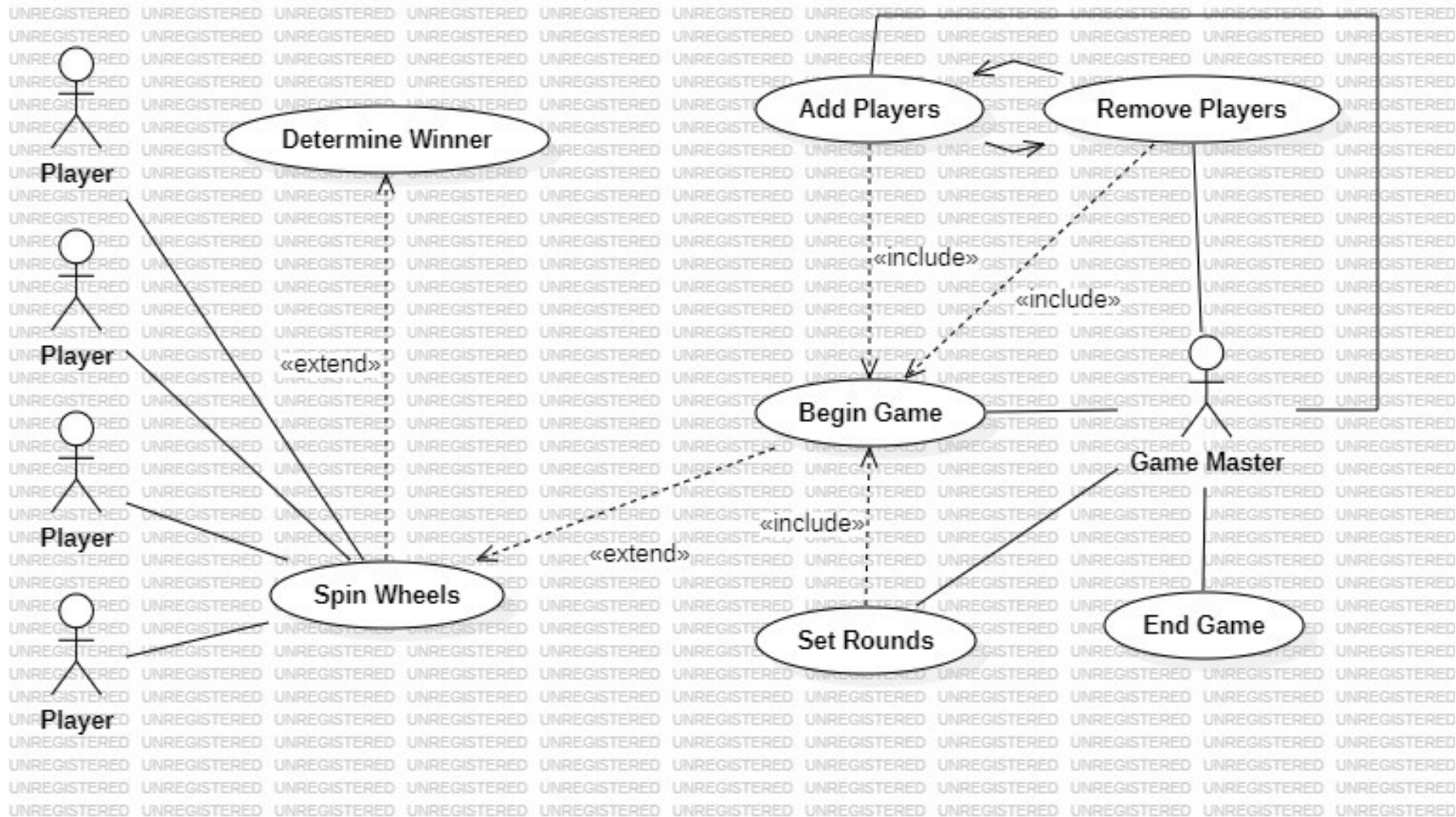
CS 3354.001 Project Report

Contents Table:

- 1. Section 1: Use Case – pg. 2**
- 2. Section 2: Class Relationships + Diagram – pg. 4**
- 3. Section 3: Sequence of Generalized Events – pg. 6**
- 4. Section 4: GUI Wireframe and Usage Guide – pg. 8**
- 5. Section 5: Raw Code Files – pg. 11**
 - a. Blitz.java – pg. 11**
 - b. Player.java – pg. 30**
 - c. Wheel.java – pg. 33**
 - d. GoodWheel.java – pg. 36**
 - e. BadWheel.java – pg. 40**
 - f. ChaosWheel.java – pg. 44**

Section 1: Use Case

Use Case	BLITZ - A Game of Chance: Standard Game Flow
Created By	Taylor Fanely-Luevano
Description	A simple party game based on random chance wheels
Actors	Game master, 4 - 8 players
Preconditions	1. There are at least 4 players participating 2. The round count is a number from 3 to 9
Postconditions	1. The player's final coin counts are counted and compared 2. The player with the highest coin count is declared the winner 3. If there is a tie, then attempt to tiebreak by spinning wheels
Flow	1. The game master adds or removes players based on participation 2. The game master sets the number of rounds to be played in the game 3. The game master begins the game 4. A player chooses to spin a wheel 5. The game master spins a wheel, and an effect is randomly chosen 6. The effect is applied to one or all players 7. 4 - 6 repeat for each player participating 8. Once all players have spun a wheel, advance a round and repeat 4 - 7 9. Repeat 4 - 8 until all rounds have been completed
Alternate Flow	4. A player may choose to forfeit their turn, in which case skip 5 and 6
	7. There is a chance the Chaos Wheel can activate - if this occurs: 1. The next player is forced to spin the Chaos Wheel 2. The Chaos Wheel lands on a random effect that applies to one or more players 3. The Chaos Wheel advances its rest state, which will eventually deactivate it
Exceptions	1. If more than 8 players are attempted to be added, then deny the operation 2. If the round count is set to be higher or lower than 9/3, then set the round count to either value 3. If less than 4 players are present when starting, the prevent the game from being able to start 4. If a player tries to spin a wheel but can't afford it, then they must choose another action 6. If a player's coin count would be reduced to less than 0, the auto-set it back to 0



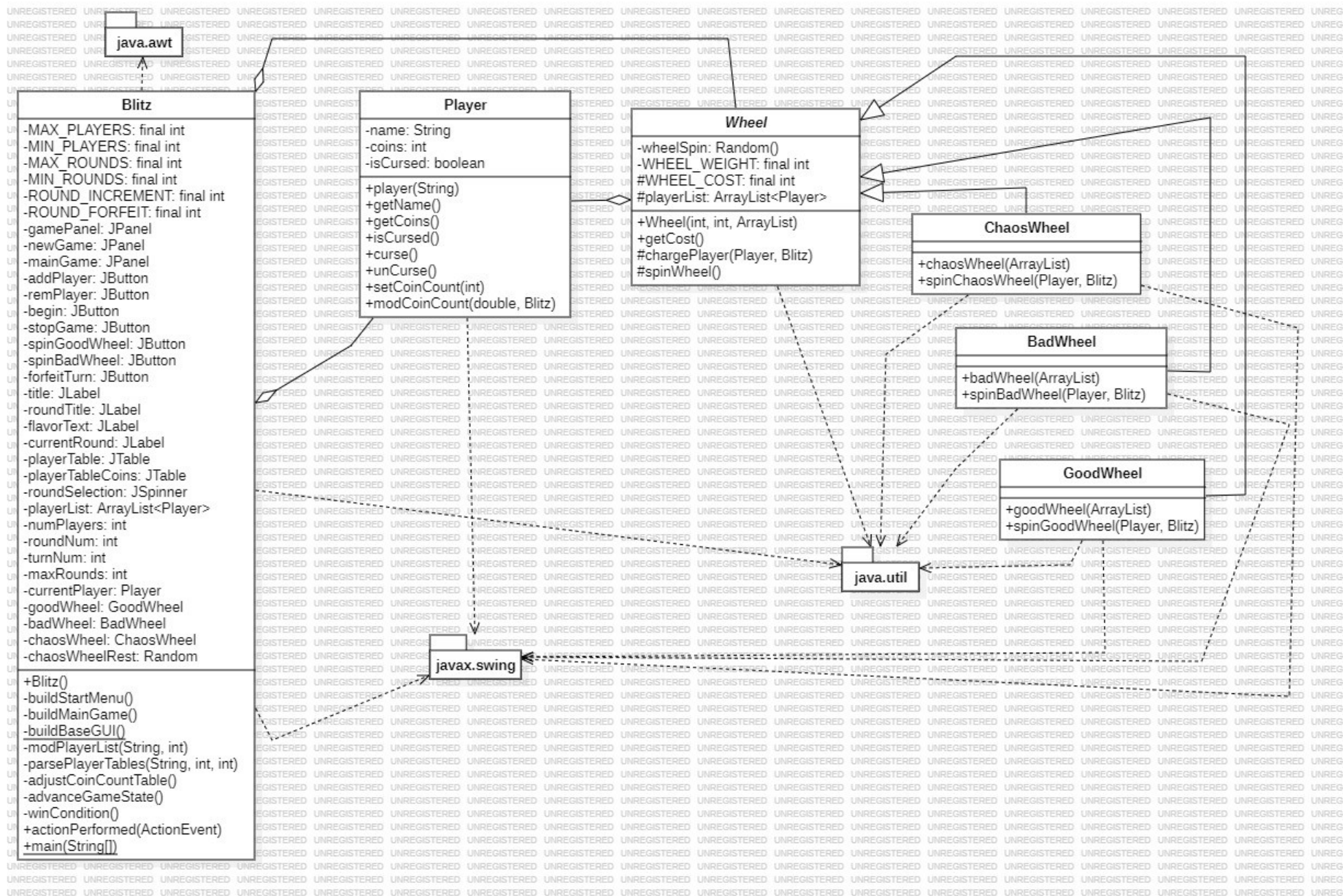
Section 2: Class Relationships + Diagram

Blitz – Main Program	
1. Store all gameplay constants 2. Build and display the GUI elements 3. Track all player related activity 4. Store all critical game elements	- Player - GoodWheel - BadWheel - ChaosWheel - playerList

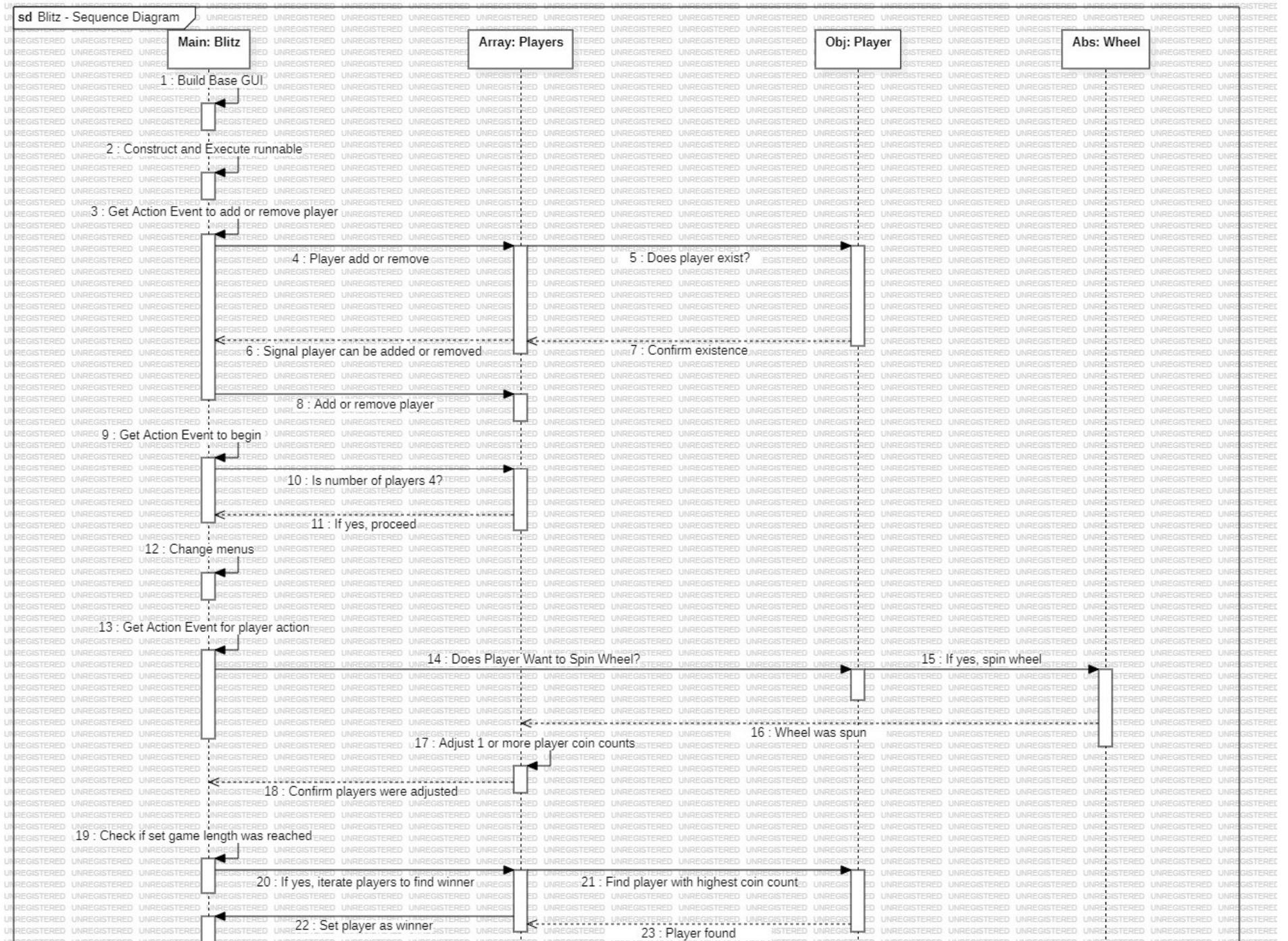
Player	
1. Stores Player information 2. Sets the state of every player depending on game actions	- Name - Coin count - isCursed

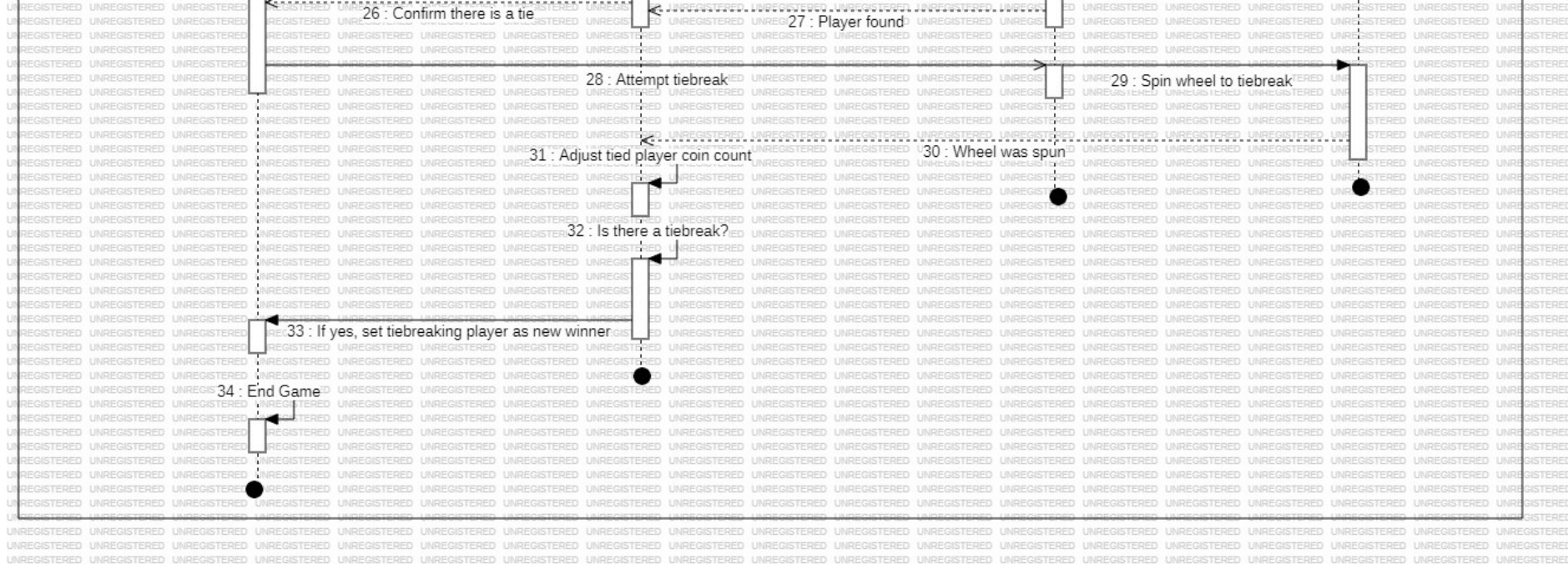
<i>Wheel</i>	
1. Abstract class that sets the default construction of each wheel 2. Sets the cost of each wheel and charges for its use 3. Allows spinning of each wheel	- wheelSpin - wheel weight - wheel cost

Good/Bad/Chaos Wheels	
1. Takes the Wheel constructor and uses their own individual values 2. Takes the spin of the wheel and has their own effects depending on where it lands	- spinGood/Bad/ChaosWheel



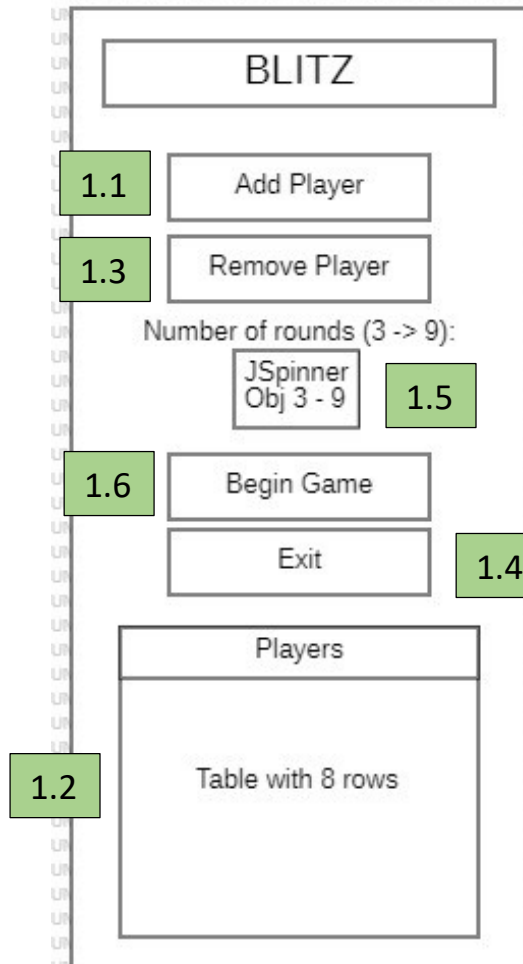
Section 3: Sequence of Generalized Events



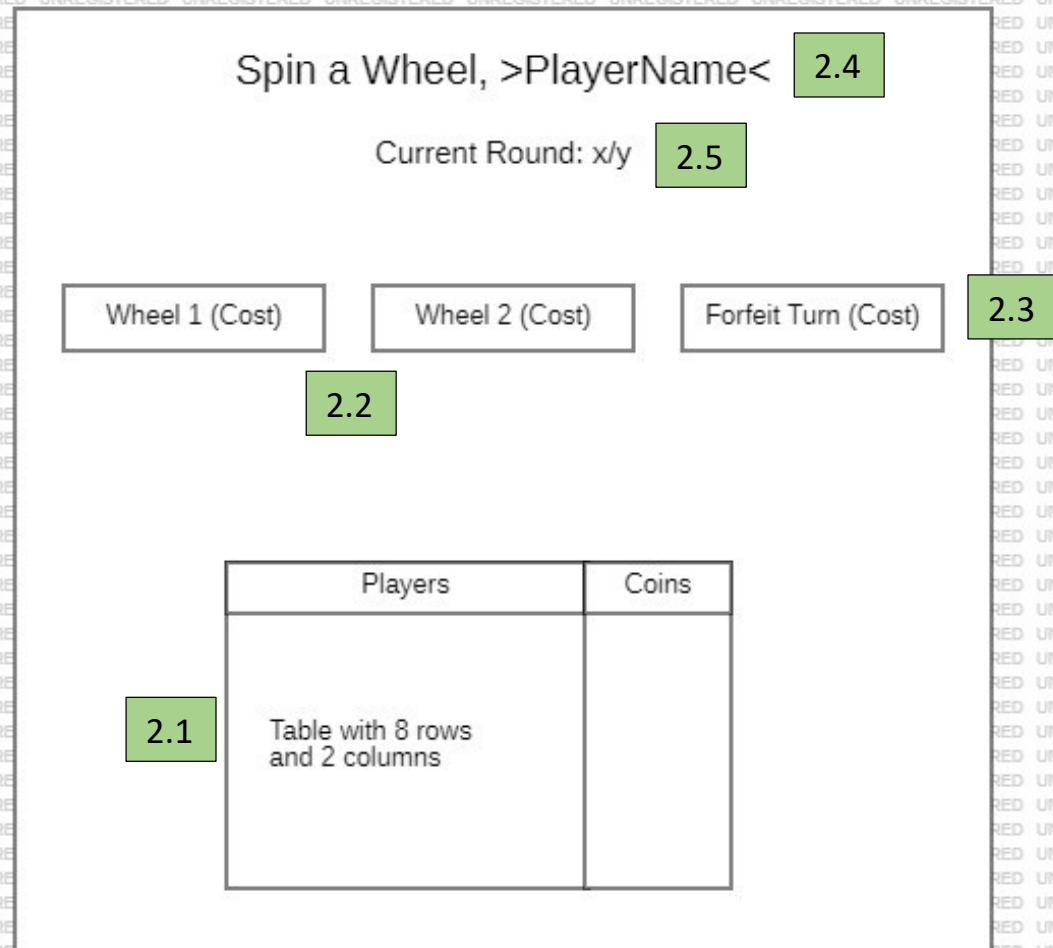


Section 4: GUI Wireframe and Usage Guide

Start Menu GUI



Main Game Menu GUI



General Usage Instructions

1. Press the “Add Player”^{1.1} button to add a participant; a text popup will appear so the name can be entered. When done, and a valid name is added (no blank names), the name will appear in the Table labeled “Players”^{1.2}
 - a. If a player wants to be removed from the game, press the “Remove Player”^{1.3} button; a text popup will appear so the name can be entered. When done, and a valid name is added (the name exists in the player list), the player will be removed.
 - b. Alternatively, if the game should need to be ended, then the “Exit”^{1.4} button can be pressed as well.
2. The JSpinner Object, labeled with “Number of Rounds”^{1.5} can adjust the number of rounds to be played before the game begins, with a number between 3 and 9.
3. To start the game, press “Begin Game”^{1.6}. The Start menu will disappear and be replaced by the main game menu.
 - a. If the number of players is lower than 4, a popup will appear stating that at least 4 players are needed to begin.
 - b. Additionally, popups will appear for trying to add more than 9 players, trying to remove players when 0 exist, and trying to add a player that has already been added.
4. Each player will be initialized with a certain amount of coins in the table labeled “Players” and “Coins”^{2.1}.
5. To spin a wheel or forfeit a round, press one of the three buttons presented; 2 are wheels to spin^{2.2}, and 1 is to forfeit^{2.3}. Once pressed, a popup will notify of the effect the selection had and the table will update.

- a. In the full program, the first wheel has a negative cost; if the player cannot afford the cost, then a popup will notify that another selection must be made.
 - b. When the Chaos Wheel advances its rest state, this will be relayed through a popup. When it needs to be spun by a player, then this will also be relayed.
6. During these processes, the name label^{2.4} will update to the name of the player whose turn it is, and the round counter^{2.5} will increment once all players in the tabel have been cycled through.
7. Once all rounds are complete, the winner will be determined; if a tie is detected, then a few popups will display an attempted tiebreak.
 - a. If the tiebreak succeeds, then the process will repeat until there is 1 true winner.
8. Once a true winner is determined, they will be delcared and the program will exit.

Section 5: Raw Code Files of Final Program

FILE 1 - Blitz.java

```
package blitz_gui;

import java.util.*;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import javax.swing.table.TableColumnModel;

/**
 * BLITZ : The Ultimate Game of Chance!
 *
 * This is a simple multiplayer game for up to 8 people that
 * intended to play much like a party game, with mostly random
 * elements that add to both the chaotic nature of such games
 * while encouraging replayability.
 *
 * @author Taylor Fanel-Luevano
 * @version 1.1
 */
public class Blitz extends JFrame implements ActionListener {
    private final int MAX_PLAYERS = 8, // player/round count related limits
        MIN_PLAYERS = 4,
        MAX_ROUNDS = 9,
        MIN_ROUNDS = 3,
        ROUND_INCREMENT = 3,
        ROUND_FORFEIT = -1; // determines the cost for forfeiting a turn

    private JPanel gamePanel = new JPanel(new GridBagLayout()), // JPanels for all GUI elements
        newGame = new JPanel(new GridBagLayout()), // each one uses a GridBag layout
        mainGame = new JPanel(new GridBagLayout());
```

```

private JButton addPlayer = new JButton("Add PLayer"), // Buttons for all player functions
    remPlayer = new JButton("Remove Player"),
    begin = new JButton("Begin Game"),
    stopGame = new JButton("Exit"),
    spinGoodWheel = new JButton(), // No labels as they are established later
    spinBadWheel = new JButton(),
    forfeitTurn = new JButton();

private JLabel title = new JLabel("BLITZ"), // Labels for all related elements
    roundTitle = new JLabel("Number of Rounds (" + MIN_ROUNDS + " -> " + MAX_ROUNDS + "):"),
    flavorText = new JLabel("Spin a wheel if you dare, %s..."), // Placeholder Labels; prevents a critical
    currentRound = new JLabel("Current Round: %d"); // bug where the table isn't sized properly

private JTable playerTable, // Tables to keep track of player activity
    playerTableCoins;

private JSpinner roundSelection; // Spinner to set the number of rounds

private ArrayList<Player> playerList = new ArrayList<Player>(); // Array that stores all player objects

private int numPlayers = 0, // Game variables that are dynamic each game
    roundNum = 1,
    turnNum = 0,
    maxRounds;

private Player currentPlayer; // Stores the player that currently has a turn

private final GoodWheel goodWheel = new GoodWheel(playerList); // Wheel objects for the game
private final BadWheel badWheel = new BadWheel(playerList);
private final ChaosWheel chaosWheel = new ChaosWheel(playerList);

private Random chaosWheelRest; // Unique random generator for the ChaosWheel

/**
 * Constructor for the main game window
 * @see Blitz#buildStartMenu()

```

```

* @see Blitz#buildMainGame()
*/
public Blitz() {
    setTitle("BLITZ");
    setResizable(false);
    add(gamePanel);

    // builds the primary game menus
    buildStartMenu();
    buildMainGame();
    mainGame.setVisible(false); // So the main game menu doesn't overlap the start menu

    GridBagConstraints l = new GridBagConstraints();
    l.insets = new Insets(10, 10, 10, 10);
    l.fill = GridBagConstraints.HORIZONTAL;
    l.gridx = 0;
    l.gridy = 0;
    gamePanel.add(newGame, l);
    l.insets = new Insets(10, 10, 10, 10);
    l.fill = GridBagConstraints.HORIZONTAL;
    l.gridx = 0;
    l.gridy = 0;
    gamePanel.add(mainGame, l);
}

/**
 * Constructs the start menu, allowing the addition and
 * removal of players and setting the number of rounds
 */
private void buildStartMenu() {
    String[] nameListHeading = {"Players"}; // String arrays to store into the table
    String[][] playerNameList = new String[MAX_PLAYERS][1];

    addPlayer.setPreferredSize(new Dimension(120, 25));
    addPlayer.addActionListener(this);
    remPlayer.setPreferredSize(new Dimension(120, 25));

```

```

remPlayer.addActionListener(this);
begin.setPreferredSize(new Dimension(120, 25));
begin.addActionListener(this);
stopGame.setPreferredSize(new Dimension(120, 25));
stopGame.addActionListener(this);

SpinnerNumberModel m = new SpinnerNumberModel(MIN_ROUNDS, MIN_ROUNDS, MAX_ROUNDS, ROUND_INCREMENT);
roundSelection = new JSpinner(m);

// playerTable stores just the names of the players
playerTable = new JTable(playerNameList, nameListHeading);
playerTable.setEnabled(false); // To prevent editing of the table
TableColumnModel t = playerTable.getColumnModel();
t.getColumn(0).setPreferredWidth(150);

title.setFont(new Font("Comic Sans MS", Font.BOLD, 18));
roundTitle.setFont(new Font("Comic Sans MS", Font.PLAIN, 14));

GridBagConstraints l = new GridBagConstraints();
l.insets = new Insets(15, 10, 15, 10); // Top portion (Title -> Round selector)
l.fill = GridBagConstraints.CENTER;
l.gridx = 0;
l.gridy = 0;
newGame.add(title, l);
l = new GridBagConstraints();
l.insets = new Insets(1, 30, 1, 30);
l.fill = GridBagConstraints.CENTER;
l.gridx = 0;
l.gridy = 1;
newGame.add(addPlayer, l);
l = new GridBagConstraints();
l.insets = new Insets(1, 30, 3, 30);
l.fill = GridBagConstraints.CENTER;
l.gridx = 0;
l.gridy = 2;
newGame.add(remPlayer, l);

```



```

l = new GridBagConstraints();
l.insets = new Insets(2, 30, 1, 30);
l.fill = GridBagConstraints.CENTER;
l.gridx = 0;
l.gridy = 3;
newGame.add(roundTitle, l);
l = new GridBagConstraints();
l.insets = new Insets(1, 30, 15, 30);
l.fill = GridBagConstraints.CENTER;
l.gridx = 0;
l.gridy = 4;
newGame.add(roundSelection, l);

l = new GridBagConstraints(); // Middle poriton (Begin and exit buttons)
l.insets = new Insets(1, 30, 1, 30);
l.fill = GridBagConstraints.CENTER;
l.gridx = 0;
l.gridy = 5;
newGame.add(begin, l);
l = new GridBagConstraints();
l.insets = new Insets(1, 30, 15, 30);
l.fill = GridBagConstraints.CENTER;
l.gridx = 0;
l.gridy = 6;
newGame.add(stopGame, l);
l = new GridBagConstraints();
l.insets = new Insets(5, 10, 0, 10);
l.fill = GridBagConstraints.CENTER;
l.gridx = 0;
l.gridy = 7;

newGame.add(playerTable.getTableHeader(), l); // Player table
l = new GridBagConstraints();
l.insets = new Insets(0, 10, 15, 10);
l.fill = GridBagConstraints.CENTER;
l.gridx = 0;

```

```

    l.gridy = 8;
    newGame.add(playerTable, 1);
}

/**
 * Builds the main game window when the start menu confirms
 * everything can begin
 * @see Wheel#getCost()
 */
private void buildMainGame() {
    String[] nameListHeading = {"Players", "Coins"}; // String arrays to store into the table
    String[][] playerStatList = new String[MAX_PLAYERS][2];
    JPanel flavorPanel = new JPanel(new GridBagLayout()), // 2 new Panels to separate the titles and buttons
        buttonPanel = new JPanel(new GridBagLayout());
    setLayout(new GridBagLayout());

    // Uses the .getCost() function of Wheel to properly display the button text
    spinGoodWheel.setText("Good Wheel (" + goodWheel.getCost() + ")");
    spinGoodWheel.setPreferredSize(new Dimension(125, 25));
    spinGoodWheel.addActionListener(this);
    spinBadWheel.setText("Bad Wheel (" + badWheel.getCost() + ")");
    spinBadWheel.setPreferredSize(new Dimension(125, 25));
    spinBadWheel.addActionListener(this);
    forfeitTurn.setText("Forfeit Turn (" + ROUND_FORFEIT + ")");
    forfeitTurn.setPreferredSize(new Dimension(125, 25));
    forfeitTurn.addActionListener(this);

    // playerTableCoins stores the name + coin counts of each player
    playerTableCoins = new JTable(playerStatList, nameListHeading);
    playerTableCoins.setEnabled(false);
    TableColumnModel t = playerTableCoins.getColumnModel();
    t.getColumn(0).setPreferredWidth(150);
    t.getColumn(1).setPreferredWidth(50);

    flavorText.setFont(new Font("Comic Sans MS", Font.BOLD, 18));
    currentRound.setFont(new Font("Comic Sans MS", Font.PLAIN, 14));
}

```

```
GridBagConstraints l = new GridBagConstraints();
l.insets = new Insets(0, 0, 20, 0); // adds the new panels to the main game panel
l.fill = GridBagConstraints.CENTER;
l.gridx = 0;
l.gridy = 0;
mainGame.add(flavorPanel, l);
l = new GridBagConstraints();
l.insets = new Insets(20, 0, 0, 0);
l.fill = GridBagConstraints.CENTER;
l.gridx = 0;
l.gridy = 1;
mainGame.add(buttonPanel, l);

l = new GridBagConstraints(); // top portion (Titles for the main game)
l.insets = new Insets(5, 0, 5, 0);
l.fill = GridBagConstraints.NORTH;
l.gridx = 1;
l.gridy = 0;
flavorPanel.add(flavorText, l);
l = new GridBagConstraints();
l.insets = new Insets(5, 0, 0, 0);
l.fill = GridBagConstraints.NORTH;
l.gridx = 1;
l.gridy = 1;
flavorPanel.add(currentRound, l);

l = new GridBagConstraints(); // middle portion (buttons to press for player action)
l.insets = new Insets(0, 20, 50, 15);
l.fill = GridBagConstraints.HORIZONTAL;
l.gridx = 0;
l.gridy = 2;
buttonPanel.add(spinGoodWheel, l);
l = new GridBagConstraints();
l.insets = new Insets(0, 20, 50, 20);
l.fill = GridBagConstraints.HORIZONTAL;
```

```

l.gridx = 1;
l.gridy = 2;
buttonPanel.add(spinBadWheel, l);
l = new GridBagConstraints();
l.insets = new Insets(0, 15, 50, 20);
l.fill = GridBagConstraints.HORIZONTAL;
l.gridx = 2;
l.gridy = 2;
buttonPanel.add(forfeitTurn, l);

l = new GridBagConstraints(); // bottom portion (playerCoinTable)
l.insets = new Insets(50, 0, 0, 0);
l.fill = GridBagConstraints.CENTER;
l.gridx = 0;
l.gridy = 2;
mainGame.add(playerTableCoins.getTableHeader(), l);
l = new GridBagConstraints();
l.insets = new Insets(0, 0, 15, 0);
l.fill = GridBagConstraints.CENTER;
l.gridx = 0;
l.gridy = 3;
mainGame.add(playerTableCoins, l);
}

/**
 * Constructs all the elements into the main GUI window
 */
private static void buildBaseGUI() {
    Blitz gameWindow = new Blitz();
    gameWindow.setDefaultCloseOperation(EXIT_ON_CLOSE);
    gameWindow.pack();
    gameWindow.setVisible(true);
    gameWindow.setLocationRelativeTo(null); // So it always centers on the screen
}

/**

```

```

* Modifies the Player array to add or remove players from it.
* @param name - player name
* @param opCode - the operation code - is the player being added (1) or removed (2)?
* @return a boolean to confirm if the player table was updated or not
* @see Blitz#parsePlayerTables(String, int, int)
*/
private boolean modPlayerList(String name, int opCode) {
    boolean hasListBeenModded = false; // boolean to return
    boolean canPlayerBeAdded = true; // boolean to check if the player can be added to the list

    // if a player is trying to be added...
    if (opCode == 1) {
        /* Check if the player name is empty
        if it isn't, then check if the player list already contains the player.
        if either check fails, then set canPlayerBeAdded to false */
        if (name.trim().isEmpty()) canPlayerBeAdded = false;
        else if (numPlayers != 0) {
            for (int i = 0; i < numPlayers; i++) {
                if (name.equals(playerList.get(i).getName())) {
                    canPlayerBeAdded = false;
                    break;
                }
            }
        }
        // If the player can be added, then attempt to add them; otherwise, deny the operation
        if (canPlayerBeAdded) {
            Player newPlayer = new Player(name);
            playerList.add(newPlayer);
            numPlayers++;
            parsePlayerTables(name, opCode, numPlayers);
            hasListBeenModded = true; // Operation was successful
        }
        // If a player is trying to be removed...
    } else if (opCode == 2) {
        // Check if the player exists in the list; if they do, then attempt to remove them
        for (int i = 0; i < numPlayers; i++) {

```

```

        if (name.equals(playerList.get(i).getName())) {
            playerList.remove(i);
            numPlayers--;
            parsePlayerTables(name, opCode, i+1);
            hasListBeenModded = true;
            break;
        }
    }
}

// If it returns true, then the Player list was successfully adjusted
return hasListBeenModded;
}

/**
 * Updates the player JTables to add or remove the names of players being
 * added/removed
 * @param name - name of the player
 * @param opCode - operation code; 1 for add, 2 for remove
 * @param playerNum - stores the index the player is to be added to/removed from
 */
private void parsePlayerTables(String name, int opCode, int playerNum) {

    // If a player is being added...
    if (opCode == 1) {
        // set the values at index playerNum of the tables to the player name.
        playerTable.setValueAt(name, playerNum - 1, 0);
        playerTableCoins.setValueAt(name, playerNum - 1, 0);
    } // If a player is being removed...
    else if (opCode == 2) {
        // set the values at index playerNum of the tables to null.
        playerTable.setValueAt(null, playerNum - 1, 0);
        playerTableCoins.setValueAt(null, playerNum - 1, 0);

        // Once removed, check if the number of players is greater than 0. If not...
        if (numPlayers != 0) {

```

```

        // Iterate each filled index slot in the tables and update the names to remove any gaps in the list
        for (int i = playerNum - 1; i < numPlayers; i++) {
            playerTable.setValueAt(playerTable.getValueAt(i+1, 0), i, 0);
            playerTableCoins.setValueAt(playerTable.getValueAt(i+1, 0), i, 0);
        }
    }
    // Finally, if the number of players is not at maximum, set the last index value to null to remove dupes
    if (numPlayers != MAX_PLAYERS) {
        playerTable.setValueAt(null, numPlayers, 0);
        playerTableCoins.setValueAt(null, numPlayers, 0);
    }
}

/**
 * Adjust the coins displayed for each player in playerTableCoins.
 * @see Player#getCoins()
 * @see Player#isCursed()
 * @see Player#getName()
 */
private void adjustCoinCountTable() {
    String coins; // stores the coins of each player

    // iterate through the Player list and update each coin value in the table
    for (int i = 0; i < numPlayers; i++) {
        coins = Integer.toString(playerList.get(i).getCoins());
        if (!playerTableCoins.getValueAt(i, 1).equals(coins))
            playerTableCoins.setValueAt(coins, i, 1);

        // If a player is cursed, then additionally add (CURSED!) to their name in the table
        if (playerList.get(i).isCursed())
            playerTableCoins.setValueAt(playerList.get(i).getName()+" (CURSED!)", i, 0);

        // Failsafe as the cursed effect text is not updated anywhere else
        else playerTableCoins.setValueAt(playerList.get(i).getName(), i, 0);
    }
}

```

```

}

/**
 * Advances to the next player that has a turn, and advances
 * the round if all players had a turn.
 * Additionally has functionality for the activation of the
 * Chaos Wheel.
 * @see Blitz#winCondition()
 * @see Blitz#adjustCoinCountTable()
 * @see Blitz#advanceGameState()
 * @see Player#getName()
 * @see ChaosWheel#isExhausted()
 * @see ChaosWheel#spinChaosWheel(Player, Blitz)
 */
private void advanceGameState() {
    int chaosAdvance; // Determines if the Chaos Wheel must be spun

    // This function assumes a player has completed their turn, so the turn num always increments first
    turnNum++;

    // If the turn num is equal to the number of players, then the round is complete and advances to the next round
    if (turnNum == numPlayers) {
        roundNum++;
        if (roundNum == maxRounds + 1) winCondition(); // If the rounds exceed the max, the game is complete
        turnNum = 0;
        currentRound.setText("Current Round: "+roundNum+"/"+maxRounds);
    }
    // Once the turn logic is complete, use turnNum to parse the index of the next player to get a turn
    currentPlayer = playerList.get(turnNum);
    flavorText.setText("Spin a wheel if you dare, "+currentPlayer.getName()+"...");

    // Since the Chaos wheel requires no input, it is handled separately to the other 2 wheels
    chaosWheelRest = new Random(); // A number between 1 and the number of players/2 is chosen
    chaosAdvance = chaosWheelRest.nextInt(numPlayers/2)+1;

    /* If the number rolled is the maximum number that can be rolled

```



```

        (i.e. 2 for a 4 player game, 4 for an 8 player) and the Chaos Wheel
        is not in its resting state, then the Chaos wheel must be spun for the next player */
        if (chaosAdvance == numPlayers/2 && !chaosWheel.isExhausted()) {
            chaosWheel.spinChaosWheel(currentPlayer, this);
            adjustCoinCountTable(); // Performs all the logic for spinning a wheel, including recursively calling the
function
            advanceGameState();
        }
    }

    /**
     * When all rounds are done, determine the winner of the game.
     * @see Player#getCoins()
     * @see Player#getName()
     * @see GoodWheel#spinGoodWheel(Player, Blitz)
     * @see Blitz#adjustCoinCountTable()
     */
    private void winCondition() {
        Player winner = playerList.get(0); // The first player in the Player list is assumed the winner initially

        // Iterates through the player list to determine who has higher coin counts
        for (Player win : playerList) {
            if (win.getCoins() > winner.getCoins()) winner = win;
        }

        // Iterate a second time to confirm any ties
        for (Player win : playerList) {

            // Tiebreak by forcing the competing winner to spin the good wheel
            if (win.getCoins() == winner.getCoins() && !win.getName().equals(winner.getName())) {
                JOptionPane.showMessageDialog(this, "Tied score by "+winner.getName()+" and "+win.getName()+"!"+"
                "\nTiebreak the winner by spinning the good wheel!");
                flavorText.setText("SUDDEN DEATH, SPIN THE WHEEL "+win.getName()+"!");
                currentRound.setText("Current Round: !!!");
                goodWheel.spinGoodWheel(win, this);
            }
        }
    }
}

```

```

        // Display if the tiebreak succeeded or not
        if (win.getCoins() > winner.getCoins())
            JOptionPane.showMessageDialog(this, "Tiebreak! "+win.getName()+" now has a higher coin count!");
        else JOptionPane.showMessageDialog(this, "Tiebreak failed "+win.getName()+"...sorry.");

        /* To take into account any unexpected outcomes (such as a losing player
        suddenly gaining enough coins to win outright or compete for the win),
        the function is recursively called */
        winCondition();
    }
}

adjustCoinCountTable(); // So the winner's coins are properly displayed
JOptionPane.showMessageDialog(this, winner.getName()+" has won the game of BLITZ! Congratulations!");
System.exit(0);
}

```

@Override

```

/**
 * The action event to handle any button usage.
 * @see Blitz#modPlayerList(String, int)
 * @see Blitz#advanceGameState()
 * @see Blitz#adjustCoinCountTable()
 * @see Player#getName()
 * @see Player#getCoins()
 * @see Player#modCoinCount(double, Blitz)
 * @see GoodWheel#spinGoodWheel(Player, Blitz)
 * @see BadWheel#spinBadWheel(Player, Blitz)
 * @see ChaosWheel#spinChaosWheel(Player, Blitz)
 * @see ChaosWheel#rest()
 */
public void actionPerformed(ActionEvent op) {
    JButton src = (JButton)op.getSource(); // Gets the operation button
    String name; // For when a player is being added/removed
    int chaosAdvance; // For when a player completes their turn

```

```

// Operation 1: Adding a player
if (src == addPlayer) {

    // First check if the player list is full
    if (numPlayers == MAX_PLAYERS) {
        JOptionPane.showMessageDialog(this, "No more players can be added.");
    } else {

        // Enter a name to be added; the null check is if the op is canceled
        name = JOptionPane.showInputDialog(this, "Enter player name");
        if (name != null) {

            // Try to add the player, and notify if they were successfully added
            if (modPlayerList(name, 1))
                JOptionPane.showMessageDialog(this, "Player added!");
            else JOptionPane.showMessageDialog(this, "Player can't be added.");
        }
    }
}

// Operation 2: Removing a player
} else if (src == remPlayer) {

    // First check if there are even players to remove
    if (numPlayers == 0) {
        JOptionPane.showMessageDialog(this, "But...there aren't any players.");
    } else {

        // Enter a name to be removed; the null check is if the op is canceled
        name = JOptionPane.showInputDialog(this, "Enter player name");
        if (name != null) {

            // Try to remove the player, and notify if they were successfully removed
            if (modPlayerList(name, 2))
                JOptionPane.showMessageDialog(this, "Player removed...");
            else JOptionPane.showMessageDialog(this, "That player doesn't exist.");
        }
    }
}

```

```

// Operation 3: Starting the game
} else if (src == begin) {
    maxRounds = (Integer)roundSelection.getValue(); // Get the desired round count from the JSpinner

    // First check if the game has the minimum amount of required players
    if (numPlayers < MIN_PLAYERS) {
        JOptionPane.showMessageDialog(this, "This game needs "+MIN_PLAYERS+" or more players to begin.");
    } else {

        /* Notify the number of rounds to be played and run logic to
        switch from the start menu to the main game menu*/
        JOptionPane.showMessageDialog(this, "The game begins! (Play for "+maxRounds+" rounds)");
        newGame.setVisible(false);
        mainGame.setVisible(true);
        pack();
        setLocationRelativeTo(null);

        /* Initialize the coin counts on playerTableCoins (the names were
        already handled by the parsePlayerTables function) */
        for (int i = 0; i < numPlayers; i++) {
            playerTableCoins.setValueAt(Integer.toString(playerList.get(i).getCoins()), i, 1);
        }

        currentPlayer = playerList.get(turnNum); // As turnNum is initialized to 0, there is no need to run
advanceGameState() yet
        flavorText.setText("Spin a wheel if you dare, "+currentPlayer.getName()+"...");
        currentRound.setText("Current Round: 1/"+maxRounds);
    }
// Operation 4: exiting the game
} else if (src == stopGame) {
    System.exit(0); // If the exit button on the start menu is pressed
} else {
    // If the op is not any of the above, it's assumed to come from the main game menu

    // Operation 5: Spin the Good Wheel
    if (src == spinGoodWheel) {

```

```

        // If the player can't afford the cost, then notify; otherwise, let them spin it
        if (currentPlayer.getCoins() < Math.abs(goodWheel.getCost()))
            JOptionPane.showMessageDialog(this, "You cannot afford to spin the good wheel
"+currentPlayer.getName());
        else {
            goodWheel.spinGoodWheel(currentPlayer, this);
            adjustCoinCountTable();
            advanceGameState();
        }
    // Operation 6 and 7: Spin the Bad Wheel or Forfeit the turn
    } else {

        // Let the player spin the Bad Wheel
        if (src == spinBadWheel) badWheel.spinBadWheel(currentPlayer, this);

        // If it wasn't the Bad Wheel, then the player is assumed to have forfeited their turn
        else {
            JOptionPane.showMessageDialog(this, currentPlayer.getName()+" has forfeited their turn.");
            currentPlayer.modCoinCount(ROUND_FORFEIT, this);
        }
        adjustCoinCountTable();
        advanceGameState();
    }

    // Logic that runs after any of the main game buttons were pressed

    chaosWheelRest = new Random(); // A number between 1 and the number of players/2 is chosen
    chaosAdvance = chaosWheelRest.nextInt(numPlayers/2)+1;

    /* If the number rolled is the maximum number that can be rolled
    (i.e. 2 for a 4 player game, 4 for an 8 player) and the Chaos Wheel
    is in its resting state, then the Chaos wheel advances its resting state */
    if (chaosWheel.isExhausted() && chaosAdvance == numPlayers/2) chaosWheel.rest(this);
}
}

```

```
/**
 * Runs the executable
 * @param args - command-line arguments
 * @see Blitz#buildBaseGUI()
 */
public static void main(String[] args) {
    SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            buildBaseGUI();
        }
    });
}
```

FILE 2 - Player.java

```
package blitz_gui;

import javax.swing.JOptionPane;

/**
 * This is the Player class that stores the info of any
 * player that participates in the game.
 *
 * @author Taylor Fanel-Luevano
 * @version 1.1
 */
public class Player {
    private String name; // Stores their name
    private int coins = 5; // Stores their coin count; initialized to 5
    private boolean isCursed = false; // Checks if they're cursed; initially false

    /**
     * Creates a player object and gives it a name.
     * @param playerName - the name to be given to the new player object
     */
    public Player(String playerName) {
        name = playerName;
    }

    /**
     * Fetches the player name.
     * @return name of the player
     */
    public String getName() {
        return name;
    }

    /**
     * Fetches the player coin count.
     */
}
```

```

    * @return coin count of the player
    */
    public int getCoins() {
        return coins;
    }

    /**
     * Checks if the player is cursed or not.
     * @return a boolean to check if player is cursed
     */
    public boolean isCursed() {
        return isCursed;
    }

    /**
     * Curses the player.
     */
    public void curse() {
        isCursed = true;
    }

    /**
     * Un-curses the player.
     */
    public void unCurse() {
        isCursed = false;
    }

    /**
     * Modifies the player's coin count directly, without involving
     * the curse status effect.
     * @param modifier - the % modifier to apply to the player's coins
     */
    public void setCoinCount(int modifier) {
        coins = modifier;
    }

```



```

/**
 * Increases or decreases the coin count based on what a spin
 * has done to the player.
 * @param coinsToMod - the number of coins to add or subtract
 * @param main - the main menu; used to display the message
 */
public void modCoinCount(double coinsToMod, Blitz main) {

    // First check if the player is cursed and if their coin gains are at least 1
    if (isCursed && coinsToMod > 0) {

        // Apply the curse effect, cutting their coin gain in half
        coins += (coinsToMod * 0.5);
        isCursed = false; // After doing this, the player is cleared of the effect
        JOptionPane.showMessageDialog(main, name+" has been cleansed at a loss of "+(int)(coinsToMod * 0.5)+" coins.");

    // If none of the above is true, simply adjust the coin count.
    } else coins += coinsToMod;

    // Prevent the coin count from going below 0
    if (coins < 0) coins = 0;
}
}

```

FILE 3 - Wheel.java

```
package blitz_gui;

import java.util.ArrayList;
import java.util.Random;

/**
 * The abstract class that gives all the wheels base functionality and
 * determines their constants.
 *
 * @author Taylor Fanel-Luevano
 * @version 1.0
 */
public abstract class Wheel {
    private Random wheelSpin = new Random(); // The seed that is generated for each wheel
    private final int WHEEL_WEIGHT; // The 'weight' of each wheel section
    protected final int WHEEL_COST; // The cost to spin the wheel
    protected ArrayList<Player> playerList; /* The wheel is given a copy of the Player list ptr so
                                             they can properly perform player-based operations */

    /**
     * Creates a wheel by setting its weight, cost, and the ptr to the Player list
     * @param wheelWeight - sets WHEEL_WEIGHT
     * @param cost - sets WHEEL_COST
     * @param players - copies the playerlist ptr
     */
    public Wheel(int wheelWeight, int cost, ArrayList<Player> players) {
        WHEEL_WEIGHT = wheelWeight;
        WHEEL_COST = cost;
        playerList = players;
    }

    /**
     * Fetches the cost of the wheel.
     * @return the wheel cost
     */
}
```

```

    */
    public int getCost() {
        return WHEEL_COST;
    }

    /**
     * ...?
     * Why did I think this was necessary? Well, I have
     * no shame, so I'm keeping this here to remind anyone
     * that you don't need to refresh a pointer to a list,
     * okay?
     * @param newList - just...does nothing basically.
     */
    /* public void refreshPlayerList(ArrayList<Player> newList) {
        playerList = newList;
    } */

    /**
     * Charges the selected player based on the cost of the wheel.
     * Can be positive, negative, or 0
     * @param player - the player to charge
     * @param menu - the main menu; used to pass to modCoinCount
     * @see Player#modCoinCount(double, Blitz)
     */
    protected void chargePlayer(Player player, Blitz menu) {
        player.modCoinCount(WHEEL_COST, menu);
    }

    /**
     * Spins the wheel and returns the number generated by
     * wheelSpin, with a min of 1 and a max of the wheel weight
     * @return The number from wheelSpin
     */
    protected int spinWheel() {
        return wheelSpin.nextInt(WHEEL_WEIGHT) + 1;
    }

```

}

FILE 4 - GoodWheel.java

```
package blitz_gui;

import java.util.ArrayList;
import javax.swing.*.*;

/**
 * The Good Wheel - it contains mostly positive
 * effects for both you...and your opponents if
 * luck isn't on your side.
 *
 * @author Taylor Fanel-Luevano
 * @version 1.2
 */
public class GoodWheel extends Wheel {

    /**
     * Build the good wheel with a weight of 5 and a cost of -2 coins
     * @param players - the Player list pointer
     * @see Wheel#Wheel(int, int, ArrayList)
     */
    public GoodWheel(ArrayList<Player> players) {
        super(5, -2, players);
    }

    /**
     * Spins the good wheel, with an effect based on the number
     * spun by spinWheel().
     * @param player - The player that spun the wheel
     * @param menu - main menu; used to display messages
     * @see Wheel#chargePlayer(Player, Blitz)
     * @see Wheel#spinWheel()
     * @see Player#modCoinCount(double, Blitz)
     * @see Player#getName()
     * @see Player#getCoins()
```

```

* @see Player#setCoinCount(int)
* @see Player#curse()
*/
public void spinGoodWheel(Player player, Blitz menu) {
    chargePlayer(player, menu); // Charge the player

    int wheelSpin = spinWheel(); // Spin the wheel to get a number
    ArrayList<Player> dynamic; // Create a new ArrayList for specific ops

    switch (wheelSpin) {

        case 1: // Outcome 1

            // Give the player a coin (essentially gives them the cost + 1)
            JOptionPane.showMessageDialog(menu, player.getName()+" has gained 1 coin. Not bad!");
            player.modCoinCount(-WHEEL_COST + 1, menu);

            break;
        case 2: // Outcome 2

            // Give the player 2 coins (essentially gives them the cost + 2)
            JOptionPane.showMessageDialog(menu, player.getName()+" has gained 2 coins. Now we're in business!");
            player.modCoinCount(-WHEEL_COST + 2, menu);

            break;
        case 3: // Outcome 3

            // Refund the player and give every other player 1 coin
            JOptionPane.showMessageDialog(menu, player.getName()+" had their coins refunded and got everyone else +1
coin.");

            player.modCoinCount(-WHEEL_COST, menu);
            for (int i = 0; i < playerList.size(); i++) {
                if (!playerList.get(i).getName().equals(player.getName()))
                    playerList.get(i).modCoinCount(1, menu);
            }
            break;
    }
}

```

```

case 4: // Outcome 4

    // Give the cost of the wheel spin over to the poorest player
    JOptionPane.showMessageDialog(menu, player.getName()+" gave the poorest player the cost of their wheel
spin.");

    dynamic = new ArrayList<Player>();

    // Add every other player to the dynamic list and determine the poorest player among them
    for (Player iterate : playerList) {
        if (!iterate.getName().equals(player.getName())) dynamic.add(iterate);
    }

    Player poorest = dynamic.get(0);

    for (Player iterate : dynamic) {
        if (iterate.getCoins() <= poorest.getCoins())
            poorest = iterate;
    }

    // Once determined, give the coins over to the poorest player
    poorest.modCoinCount(-WHEEL_COST, menu);

    break;
case 5: // Outcome 5

    // Curse the richest player and cut their coins in half
    JOptionPane.showMessageDialog(menu, player.getName()+" cursed the richest player and cut their coins in
half!");

    dynamic = new ArrayList<Player>();

    // Add every other player to the dynamic list and determine the richest player among them
    for (Player iterate : playerList) {
        if (!iterate.getName().equals(player.getName())) dynamic.add(iterate);
    }

    Player richest = dynamic.get(0);

```

```
    for (Player iterate : dynamic) {  
        if (iterate.getCoins() >= richest.getCoins())  
            richest = iterate;  
    }  
  
    // Once determined, halve the coins and curse the richest player  
    richest.setCoinCoint((int)(richest.getCoins() * 0.5));  
    richest.curse();  
  
    break;  
}  
}  
}
```


FILE 5 - BadWheel.java

```
package blitz_gui;

import java.util.ArrayList;
import javax.swing.*.*;

/**
 * The Bad Wheel - it gives you coins right off the
 * bat, but it may come at a stronger price - for you,
 * or for your opponents.
 *
 * @author Taylor Fanel-Luevano
 * @version 1.2
 */
public class BadWheel extends Wheel {

    /**
     * Build the bad wheel with a weight of 5 and a 'cost' of +2 coins
     * @param players - the Player list pointer
     * @see Wheel#Wheel(int, int, ArrayList)
     */
    public BadWheel(ArrayList<Player> players) {
        super(5, 2, players);
    }

    /**
     * Spins the bad wheel, with an effect based on the number
     * spun by spinWheel().
     * @param player - The player that spun the wheel
     * @param menu - main menu; used to display messages
     * @see Wheel#chargePlayer(Player, Blitz)
     * @see Wheel#spinWheel()
     * @see Player#modCoinCount(double, Blitz)
     * @see Player#getName()
     * @see Player#getCoins()
```

```

* @see Player#curse()
*/
public void spinBadWheel(Player player, Blitz menu) {
    int prevCoinCount = player.getCoins(); /* Since the curse effect applies to the coins gained by
                                             spinning the bad wheel, it's important to keep track of
                                             what the previous coin count of the player was */

    chargePlayer(player, menu); // Charge the player

    int wheelSpin = spinWheel(); // Spin the wheel to get a number
    ArrayList<Player> dynamic; // Create a new ArrayList for specific ops

    switch (wheelSpin) {

        case 1: // Outcome 1

            // Subtract a coin (net gain overall)
            JOptionPane.showMessageDialog(menu, player.getName()+" has lost a coin. Still a net gain, not bad!");
            player.modCoinCount(-1, menu);

            break;

        case 2: // Outcome 2

            // Subtract the earnings + 1
            JOptionPane.showMessageDialog(menu, player.getName()+" has lost their earnings + 1 coin. Better luck next
time.");
            player.modCoinCount(-(player.getCoins() - prevCoinCount) - 1, menu);

            break;

        case 3: // Outcome 3

            // Subtract the earnings and curse the player
            JOptionPane.showMessageDialog(menu, player.getName()+" lost their earnings and have been cursed!");
            player.modCoinCount(-(player.getCoins() - prevCoinCount), menu);

```

```
player.curse();
```

```
break;
```

```
case 4: // Outcome 4
```

```
// Steal 2 coins from the richest player
```

```
JOptionPane.showMessageDialog(menu, player.getName()+" stole 2 coins from the richest player!");
```

```
dynamic = new ArrayList<Player>();
```

```
// Add every other player to the dynamic list and determine the richest player among them
```

```
for (Player iterate : playerList) {
```

```
    if (!iterate.getName().equals(player.getName())) dynamic.add(iterate);
```

```
}
```

```
Player richest = dynamic.get(0);
```

```
for (Player iterate : dynamic) {
```

```
    if (iterate.getCoins() >= richest.getCoins())
```

```
        richest = iterate;
```

```
}
```

```
// Once determined, take the coins from the richest player
```

```
player.modCoinCount(2, menu);
```

```
richest.modCoinCount(-2, menu);
```

```
break;
```

```
case 5: // Outcome 5
```

```
// Take the earnings and give it over to the poorest player
```

```
JOptionPane.showMessageDialog(menu, player.getName()+"'s earnings have been stolen by the poorest player!");
```

```
dynamic = new ArrayList<Player>();
```

```
// Add every other player to the dynamic list and determine the poorest player among them
```

```
for (Player iterate : playerList) {
```

```

        if (!iterate.getName().equals(player.getName())) dynamic.add(iterate);
    }

    Player poorest = dynamic.get(0);

    for (Player iterate : dynamic) {
        if (iterate.getCoins() <= poorest.getCoins())
            poorest = iterate;
    }

    // Once determined, remove the coins and give them to the poorest player
    poorest.modCoinCount(player.getCoins() - prevCoinCount, menu);
    player.modCoinCount(-(player.getCoins() - prevCoinCount), menu);

    break;
}
}
}

```

FILE 6 - ChaosWheel.java

```
package blitz_gui;

import java.util.ArrayList;
import javax.swing.*.*;

/**
 * The Chaos Wheel - If it awakens during a turn, there
 * is always the risk of a player being forced to spin
 * it. This chaos is a little limited, but no less nerve
 * wracking.
 *
 * @author Taylor Fanel-Luevano
 * @version 1.1
 */
public class ChaosWheel extends Wheel {

    private boolean exhausted = true; // Checks if the Chaos Wheel is ready to spin
    private int fatigue = 3; // Wheel fatigue; if at 0, then the Chaos Wheel is ready to spin

    /**
     * Build the bad wheel with a weight of 5 and a cost
     * of 0 coins, since spinning it is involuntary.
     * @param players - the Player list pointer
     * @see Wheel#Wheel(int, int, ArrayList)
     */
    public ChaosWheel(ArrayList<Player> players) {
        super(5, 0, players);
    }

    /**
     * Spins the bad wheel, with an effect based on the number
     * spun by spinWheel().
     * @param player - The player that spun the wheel
     * @param menu - main menu; used to display messages
     */
}
```

```

* @see Wheel#spinWheel()
* @see Player#modCoinCount(double, Blitz)
* @see Player#getName()
* @see Player#getCoins()
* @see Player#setCoinCount(int)
* @see Player#curse()
* @see Player#unCurse()
*/
public void spinChaosWheel(Player player, Blitz menu) {

    // Announce the incoming spin
    JOptionPane.showMessageDialog(menu, player.getName()+" has been forced to spin the Chaos Wheel!");

    int wheelSpin = spinWheel(); /* Spin the wheel to get a number
                                   (since the cost is 0, chargePlayer() would do nothing) */

    switch (wheelSpin) {

        case 1: // Outcome 1

            // Do nothing
            JOptionPane.showMessageDialog(menu, player.getName()+" has had...nothing happen to them."+
                                           "\nChaos includes nothing happening, you know.");

            break;

        case 2: // Outcome 2

            // Force another Chaos Wheel spin
            JOptionPane.showMessageDialog(menu, player.getName()+" has...to spin the wheel again."+
                                           "\nWho knows, you might get stuck in a loop!");

            spinChaosWheel(player, menu);

            break;

        case 3: // Outcome 3

```

```

        // Swap the curse values of each player
        JOptionPane.showMessageDialog(menu, player.getName()+" un-cursed all cursed and cursed all not cursed."+
            "\nIt's cursed-a-geddon! Or something, idk.");

        for (int i = 0; i < playerList.size(); i++) {
            if (playerList.get(i).isCursed()) playerList.get(i).unCurse();
            else playerList.get(i).curse();
        }

        break;

    case 4: // Outcome 4

        // Reduce the player's coin count to 1
        JOptionPane.showMessageDialog(menu, player.getName()+"'s finger slipped and lost all of their coins but 1."+
            "\n...sorry?");

        player.setCoinCount(1);

        break;

    case 5: // Outcome 5

        // Give the player 3 coins
        JOptionPane.showMessageDialog(menu, player.getName()+" had defied all odds and gained +3 coins! Lucky
you.");

        player.modCoinCount(3, menu);
        break;
    }

    fatigue++; // When spun, increase the wheel's fatigue

    // Once fatigue is 3, set the Wheel to not be spun
    if (fatigue == 3) {
        JOptionPane.showMessageDialog(menu, "The Chaos wheel grows weary and wains...");
        exhausted = true;
    }
}

```

```

}

/**
 * Advances the Chaos Wheel's rest stage by removing fatigue.
 * @param menu - main menu, for displaying messages
 */
public void rest(Blitz menu) {
    fatigue--; // Assumes the Chaos Wheel is advancing its rest stage, so decrement fatigue

    // Display messages based on the state of fatigue
    if (fatigue == 1) JOptionPane.showMessageDialog(menu, "The Chaos wheel stirs...");
    else if (fatigue == 0) {
        JOptionPane.showMessageDialog(menu, "The Chaos wheel rattles to life...");
        exhausted = false; // Once fatigue is 0, set the Wheel as ready to spin
    }
}

/**
 * Checks if the Chaos Wheel is currently
 * unable to be spun.
 * @return if the wheel can be spun
 */
public boolean isExhausted() {
    return exhausted;
}
}

```